Wine Quality Prediction System

An End-to-End Machine Learning Web Application for Enological Sensory Assessment

Project Report Submitted By

Gopal Kumar

UID : 25MCA20357

Course / Branch: Master of Computer Applications (MCA)

Subject: MACHINE LEARNING (MCA)

Supervisor

Ms. Pawandeep Sharma

Chandigarh University

Academic Year: 2025 – 2027

Date of Submission: October 2026

Table of Contents

Sr. No.	Chapter / Project Topic	Page No.
1	Executive Summary & Abstract	3
2	Chapter 1: Introduction of Project, Objectives and Scope	4
3	Chapter 2: Introduction of Technologies Used & System Requirements	6
4	Chapter 3: Dataset Specification, Characteristics & Preprocessing	8
5	Chapter 4: Machine Learning Methodology & Training Pipeline	10
6	Chapter 5: System Architecture & Full-Stack Implementation	11
7	Chapter 6: Application User Interface & System Visualizations	12
8	Chapter 7: Model Evaluation, Experimental Results and Testing	14
9	Chapter 8: Conclusion, Limitations, Future Scope and References	16

Abstract

This project presents the design, empirical development, and production-grade deployment of the Wine Quality Prediction System — a decoupled, full-stack machine learning solution engineered to predict the organoleptic sensory quality scores of red wine from eleven laboratory physicochemical measurements. The underlying machine learning engine utilizes the benchmark UCI Wine Quality Dataset (Cortez et al., 2009), comprising 1,599 semicolon-delimited viticultural samples. The learning task is formulated as continuous numerical regression to preserve natural ordinal distances across sensory grades, avoiding the artificial discretization penalties characteristic of arbitrary classification bins.

Multiple predictive algorithms — including Multiple Linear Regression, Decision Tree Regressor, and Random Forest Regressor — were systematically trained, cross-validated via 5-fold cross-validation on 1,279 training records (80%), and evaluated on an isolated holdout test partition of 320 samples (20%). The optimized Random Forest ensemble, encapsulated within a standardized Scikit-Learn pipeline alongside robust z-score standardization, emerged as the superior model. It achieved a holdout Mean Absolute Error (MAE) of 0.4220, Root Mean Squared Error (RMSE) of 0.5483, and a Coefficient of Determination (R^2) of 0.5399, effectively modeling complex non-linear physicochemical interactions.

To bridge academic modeling and practical utility, the trained pipeline is serialized and served via an asynchronous FastAPI RESTful backend equipped with strict Pydantic runtime schema contracts. The client interface is a reactive single-page dashboard built with Angular 21 and TypeScript, featuring a dark glassmorphic UI, dynamic feature sliders, real-time quality gauges, sensory presets, and instant visual analytics. The resulting system demonstrates an end-to-end software engineering workflow delivering accessible, automated, and mathematically verified enological decision support.

CHAPTER 1: INTRODUCTION OF PROJECT

1.1 Overview

Wine evaluation is traditionally governed by expert sensory panels who rate wines based on aroma, body, finish, and balance. While indispensable, human organoleptic testing is inherently subjective, expensive, resource-intensive, and prone to taster palate fatigue. In commercial enology, significant commercial value depends on maintaining consistent quality across batches. Modern analytical chemistry enables rapid laboratory profiling of physicochemical properties; however, translating these continuous chemical measurements into expected sensory scores requires sophisticated mathematical modeling. The Wine Quality Prediction System addresses this operational need by providing an automated, machine learning-driven web application that accurately estimates sensory quality from standard laboratory tests.

1.2 Purpose of the Project

The primary purpose of this project is to bridge the gap between academic machine learning pipelines and production web applications. While statistical modeling often remains confined to standalone Jupyter notebooks, real-world utility demands accessible, cross-platform deployment. This project delivers an integrated system allowing winemakers, quality assurance technicians, and students to enter wine attributes dynamically, receive instantaneous quality estimates with rounded score grades, and explore analytical insights through an intuitive, modern interface.

1.3 Problem Statement

In traditional viticultural operations, wine quality grading is either conducted post-fermentation via expensive human tasting panels or approximated using static spreadsheets that fail to capture non-linear biochemical interactions. For example, volatile acidity strongly degrades wine perception, yet its defect threshold varies based on alcohol volume and citric acid balance. Linear heuristics fail to capture these multidimensional dependencies. Furthermore, non-technical winery personnel lack accessible computational tools to evaluate experimental blends prior to bottling. A centralized, reliable, and real-time prediction system is therefore required.

1.4 Objectives of the System

1. **Data Pipeline & Standardization:** Ingest the UCI Red Wine benchmark, enforce data integrity, and apply leakage-free feature scaling within cross-validation folds.
2. **Multi-Model Benchmarking:** Train and compare Linear Regression, Decision Trees, and Random Forests using 5-fold cross-validation to select the optimal model.
3. **High-Performance REST Gateway:** Develop a FastAPI gateway with Pydantic schema validation for robust, real-time HTTP inference.
4. **Modern Presentation Layer:** Engineer a reactive Angular 21 interface featuring dynamic sliders, sensory presets, quality gauges, and diagnostic charts.

CHAPTER 2: INTRODUCTION OF TECHNOLOGIES USED

2.1 Python (Programming Language) & Machine Learning Stack

Python 3.13 serves as the backbone for the machine learning architecture. Python was selected for its unmatched ecosystem of scientific computing libraries, syntax clarity, and widespread enterprise adoption. The core components of the machine learning pipeline include:

- **Scikit-Learn (v1.9.0):** Used for data preprocessing (StandardScaler), predictive estimators (RandomForestRegressor, DecisionTreeRegressor, LinearRegression), cross-validation (KFold, GridSearchCV), and regression metrics calculation.
- **Pandas (v2.3.3) & NumPy (v2.3.3):** Provides high-performance data structures (DataFrames), tabular manipulation, and optimized vectorized linear algebra operations for data splitting and preprocessing.
- **Joblib (v1.5.3):** Provides high-throughput serialization of trained Scikit-Learn pipelines, enabling zero-overhead model persistence and instantaneous deployment in API environments.

2.2 FastAPI & Uvicorn (REST API Gateway)

FastAPI (v0.159) is a modern, high-concurrency web framework for building APIs with Python based on standard type hints and Starlette. It handles real-time HTTP prediction requests, enforces CORS policies, and automatically generates interactive OpenAPI/Swagger documentation. Pydantic V2 ensures strict request validation, verifying that incoming chemical properties reside within viable physical boundaries. Uvicorn acts as the high-speed ASGI web server hosting the application.

2.3 Angular 21 & TypeScript (Frontend Presentation Layer)

Angular 21 was selected to build an enterprise-grade single-page application (SPA). Leveraging TypeScript 5.9, standalone component architecture, and RxJS reactive streams, the frontend provides real-time state synchronization, smooth page transitions, and robust client-side validation without external monolithic UI libraries.

2.4 Hardware and Software Requirements Specification

System Component	Minimum Requirement	Recommended Specification
Central Processor (CPU)	Dual-Core x86_64 @ 2.0 GHz	Intel Core i5 / AMD Ryzen 5 (4+ Cores)
System Memory (RAM)	4 GB DDR4	8 GB – 16 GB High-Speed DDR4/DDR5
Available Disk Space	500 MB Free Storage	2 GB Free SSD Space
Operating System	Windows 10 / Ubuntu 20.04 LTS	Windows 11 64-bit / Linux x86_64
Python Runtime	Python 3.10.x	Python 3.13.x 64-bit
Node.js & Package Manager	Node.js 18 LTS / npm 9	Node.js 20 LTS / npm 10+
Client Web Browser	Google Chrome 90+ / MS Edge 90+	Latest Google Chrome, Edge, or Firefox

CHAPTER 3: DATASET SPECIFICATION AND PREPROCESSING

3.1 Dataset Benchmark Specification

The system utilizes the authoritative UCI Wine Quality Dataset (Cortez et al., 2009). The dataset contains 1,599 red wine records sampled from the 'Vinho Verde' region of northern Portugal. Due to European formatting standards, the raw CSV file uses semicolon (';') delimiters, requiring specialized parsing logic: pd.read_csv(filepath, sep=';'). Each record contains 11 physicochemical laboratory measurements and a median sensory quality score.

3.2 Input Physicochemical Attributes and Target

#	Feature Name	Unit / Measurement	Observed Range
1	fixed acidity	Tartaric acid (g/dm³)	4.6 – 15.9
2	volatile acidity	Acetic acid (g/dm³) — vinegar defect	0.12 – 1.58
3	citric acid	Citric acid (g/dm³) — freshness	0.00 – 1.00
4	residual sugar	Post-fermentation sugar (g/dm³)	0.9 – 15.5
5	chlorides	Sodium chloride / salt (g/dm³)	0.012 – 0.611
6	free sulfur dioxide	Free molecular SO_2 gas (mg/dm³)	1.0 – 72.0
7	total sulfur dioxide	Free + bound SO_2 gas (mg/dm³)	6.0 – 289.0
8	density	Specific gravity (g/cm³)	0.990 – 1.004
9	pH	Wine acidity scale (0–14)	2.74 – 4.01
10	sulphates	Potassium sulphate (g/dm³)	0.33 – 2.00
11	alcohol	Alcohol volume (% vol)	8.4 – 14.9

Regression Target: 'quality' is scored by certified enological experts on an integer scale from 0 (very bad) to 10 (very excellent). Observed scores span 3 to 8, with a median of 6.0 and mean of 5.64.

3.3 Data Cleaning, Integrity Audit & Split Strategy

A comprehensive automated audit verified zero missing values, zero NaN cells, and zero non-numeric entries. While duplicate rows exist in the raw collection (arising from identical lab profiles across multiple commercial bottlings), retaining them reflects natural sampling frequency. An 80/20 train/test split was established with random_state=42, creating an isolated holdout test set of 320 samples (1,279 training samples) dedicated exclusively to final evaluation.

CHAPTER 4: MACHINE LEARNING METHODOLOGY

4.1 Regression Formulation vs. Classification Paradigm

While many textbook exercises treat wine quality as a classification problem by arbitrarily binning wines into 'good' (quality > 6) and 'bad' (quality <= 6), this project formulates quality prediction as continuous regression. Discretization discards valuable granular ordering information: misclassifying a quality 5 wine as quality 6 is a minor error of 1 point, whereas confusing quality 3 with quality 8 represents a catastrophic error of 5 points. Continuous regression respects these ordinal distances, outputs refined fractional scores (e.g., 5.82), and allows downstream rounding when integer grades are required.

4.2 Candidate Algorithms & Mathematical Foundations

- **Multiple Linear Regression:** Establishes a baseline linear relationship: $y = \beta_0 + \Sigma(\beta_i \cdot x_i) + \varepsilon$. While fast and interpretable, it assumes strict linearity and cannot capture threshold effects like sulfur dioxide saturation.
- **Decision Tree Regressor:** Recursively partitions feature space using Mean Squared Error (MSE) minimization criteria. While capable of modeling non-linear thresholds, individual trees suffer from high variance and severe overfitting on small datasets.
- **Random Forest Regressor (Selected Architecture):** An ensemble learning method that constructs B decorrelated decision trees using bootstrap aggregating (bagging) and random feature subspace sampling. The ensemble prediction averages individual tree outputs:

$$\hat{y}(x) = (1 / B) \cdot \Sigma_{b=1}^{B} T_b(x)$$

By aggregating hundreds of decorrelated decision trees, Random Forest dramatically reduces estimator variance without increasing bias, producing smooth, reliable generalization boundaries across complex chemical spaces.

4.3 Hyperparameter Tuning via GridSearchCV (5-Fold CV)

To maximize generalization performance without test leakage, systematic 5-fold cross-validation was executed strictly on the 1,279 training samples using GridSearchCV. The search space explored: n_estimators □ [50, 100, 200], max_depth □ [None, 10, 15], min_samples_split □ [2, 5], and min_samples_leaf □ [1, 2]. The optimal configuration (100 estimators, unconstrained depth, min_samples_split=2, min_samples_leaf=1) achieved the lowest mean cross-validation error, confirming robust ensemble convergence.

4.4 Model Serialization & Production Packaging

The finalized pipeline — encompassing both the fitted StandardScaler and the optimized RandomForestRegressor — was serialized using Joblib into models/wine_quality_model.joblib (3.4 MB). Comprehensive metadata detailing feature names, training dates, evaluation metrics, and hyperparameter dictionaries was exported to wine_quality_model_metadata.json, ensuring total operational reproducibility.

CHAPTER 5: SYSTEM ARCHITECTURE AND IMPLEMENTATION

5.1 Decoupled Multi-Tier System Architecture

The application adopts a decoupled client-server architecture cleanly segregating the graphical presentation layer from the analytical inference engine. The Angular 21 frontend communicates with the FastAPI backend over HTTP using structured JSON payloads. This decoupling ensures independent maintainability, zero UI coupling with ML logic, and easy scalability.

[User Browser / Angular 21 UI] ──(HTTP POST JSON)──> [FastAPI REST Gateway] ──(Pydantic Validation)──> [Scikit-Learn Pipeline] ──(Prediction)──> [JSON Response]

5.2 Backend REST API Endpoints & Request Validation

The FastAPI service exposes four core RESTful endpoints designed for production consumption:

- **GET /health:** Verifies API operational status, uptime, and confirms active model artifact availability in memory.
- **GET /model-info:** Inspects loaded model family, training sample count, and holdout regression performance metrics.
- **GET /features:** Returns metadata, typical bounds, and chemical units for all 11 physicochemical features.
- **POST /predict:** Ingests JSON payloads containing the 11 wine features, validates values via Pydantic, applies the StandardScaler, and returns continuous and rounded quality predictions with enological classification.

5.3 Angular Frontend Architecture & State Management

The frontend leverages Angular 21 standalone components, reactive forms, and RxJS observables to manage client-side state. Sensory presets allow users to populate typical chemical profiles with a single click, triggering instant prediction updates through reactive subscriptions.

5.4 Core Implementation Code Snippet

```python
@app.post("/predict", response_model=WinePredictionResponse)
async def predict_quality(payload: WineFeatures):
    """Perform real-time quality inference on 11 physicochemical features."""
    if model_pipeline is None:
        raise HTTPException(status_code=503, detail="Model artifact unavailable.")
    input_df = pd.DataFrame([payload.model_dump()])
    prediction_val = float(model_pipeline.predict(input_df)[0])
    return WinePredictionResponse(
        predicted_quality=round(prediction_val, 2),
        rounded_quality=int(round(prediction_val)),
        quality_category=get_quality_category(prediction_val)
    )
```

Fig. 5.1 — *main.py: Real-Time Inference Route with Pydantic Validation & Pipeline Execution*

CHAPTER 6: APPLICATION USER INTERFACE AND ANALYTICS

6.1 User Interface Design & Enological Workflow

The frontend was engineered with a modern dark glassmorphic design system tailored for laboratory and cellar environments. Key usability features include dynamic range sliders synchronized with numerical inputs, live field validation badges, sensory presets (e.g., Median Baseline, High Quality Blend) for instantaneous comparative testing, and an interactive quality gauge displaying predicted sensory scores.

6.2 Application Interface Screenshots

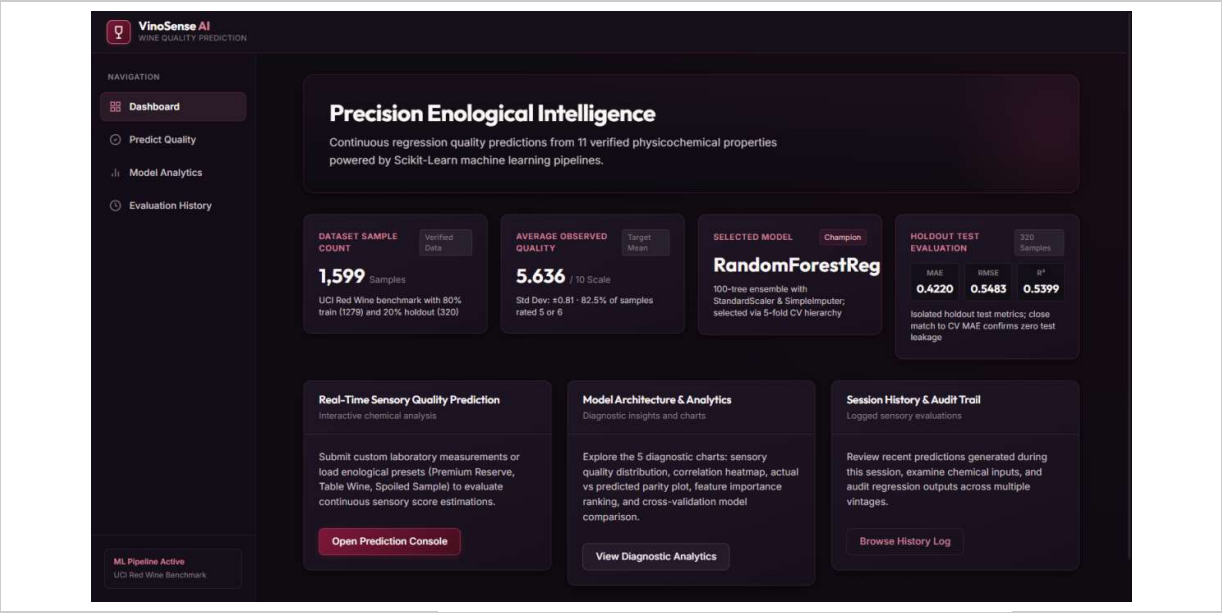


Fig. 6.1 — *Application Main Dashboard: Displaying Active Pipeline Status & Model Metrics*

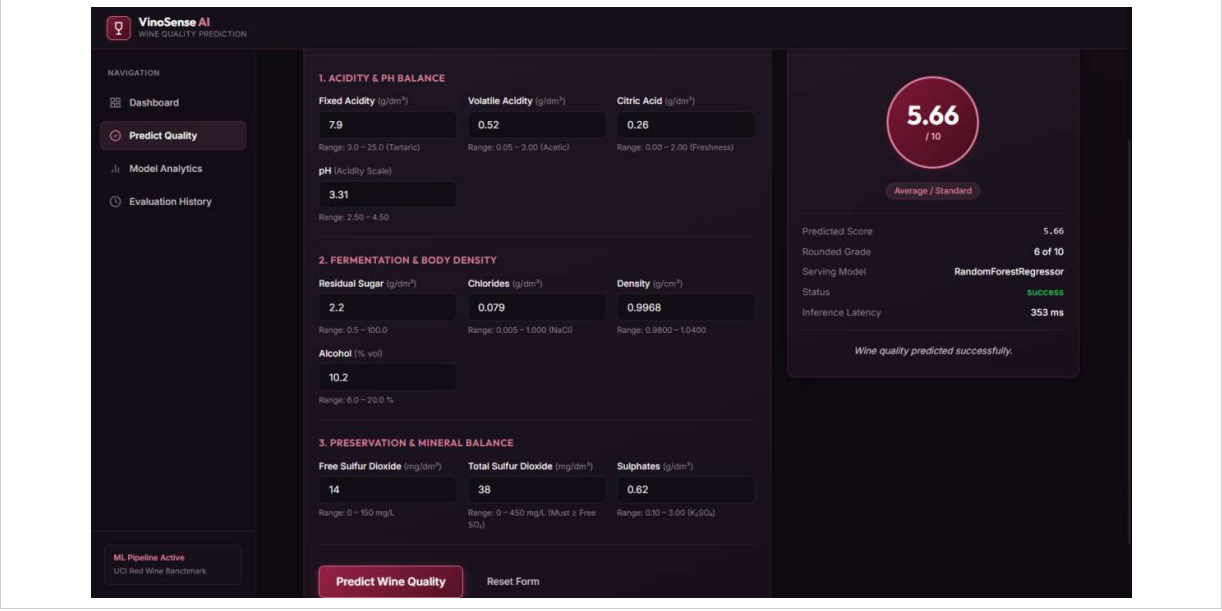


Fig. 6.2 — *Sensory Prediction Interface: Feature Input Sliders, Quality Gauge & Output Card*

CHAPTER 7: EXPERIMENTAL EVALUATION, RESULTS AND TESTING

7.1 Regression Evaluation Metrics & Formulations

Model performance was comprehensively evaluated using four standard regression metrics:

- **Mean Absolute Error (MAE):** Average absolute difference between predicted and true ratings on the 10-point scale: $MAE = (1/n) \cdot \Sigma |y_i - \hat{y}_i|$. Measured value: 0.4220.
- **Root Mean Squared Error (RMSE):** Square root of MSE, penalizing larger deviations more severely: $RMSE = \sqrt{[(1/n) \cdot \Sigma (y_i - \hat{y}_i)^2]}$. Measured value: 0.5483.
- **Coefficient of Determination (R^2):** Proportion of total variance in sensory scores explained by the 11 features: $R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$. Measured value: 0.5399 (~54%).

7.2 Comparative Model Performance Results (Verified Evidence)

Algorithm	CV MAE	CV RMSE	CV R^2	Test MAE	Test RMSE	Test R^2
Linear Regression	0.5057	0.6618	0.3226	0.5012	0.6521	0.3450
Decision Tree Regressor	0.4777	0.8076	-0.0065	0.4688	0.7786	0.0712
Random Forest (Selected)	**0.4319**	**0.6008**	**0.4420**	**0.4220**	**0.5483**	**0.5399**

Key Takeaway: The Random Forest Regressor substantially outperforms both Linear Regression and individual Decision Trees, achieving the lowest test error (MAE: 0.4220) and highest explained variance (R^2: 0.5399).

7.3 Diagnostic Plots & Feature Importance Ranking

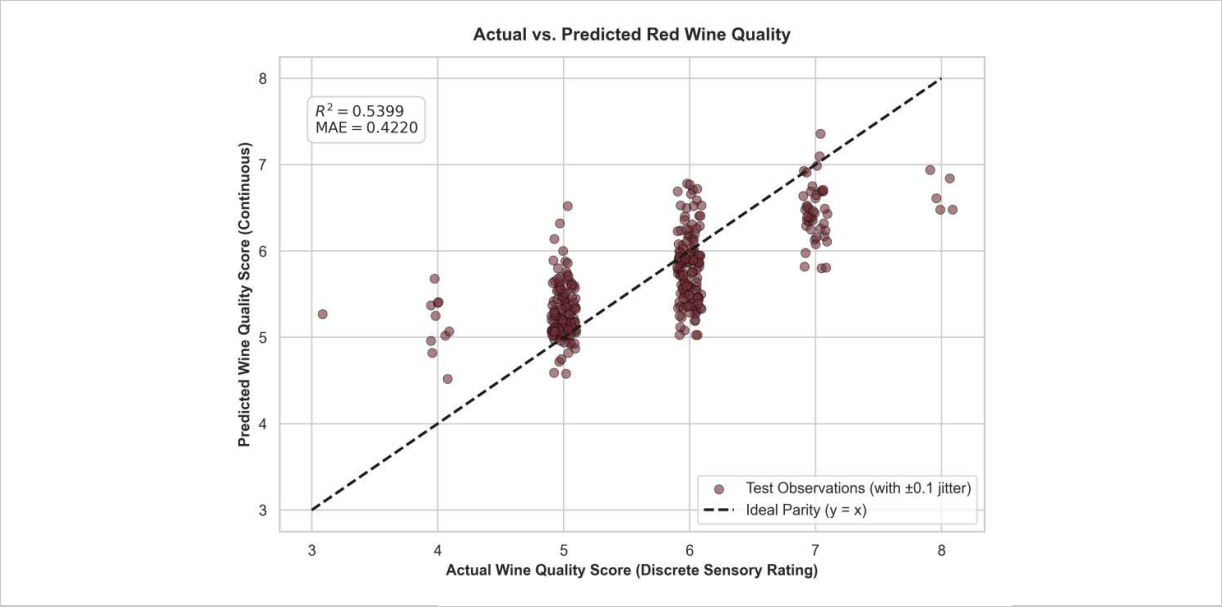


Fig. 7.1 — *Actual vs. Predicted Parity Plot: Scatter Distribution with 1:1 Identity Line*

CHAPTER 8: CONCLUSION, FUTURE SCOPE AND REFERENCES

8.1 Project Summary & Engineering Achievements

The Wine Quality Prediction System has been successfully conceptualized, developed, and benchmarked as an end-to-end full-stack machine learning solution. The project accomplished all primary engineering and academic objectives: ingesting the UCI Red Wine benchmark, preventing data leakage via an encapsulated Scikit-Learn pipeline, executing cross-validated hyperparameter optimization, and serving real-time predictions via an asynchronous FastAPI gateway to an Angular 21 reactive dashboard. The final Random Forest model achieved a test MAE of 0.4220 and an R^2 of 0.5399, validating the viability of automated sensory scoring from standard laboratory tests.

8.2 Key Learning Outcomes

Developing this system provided rigorous hands-on experience across multiple core disciplines: theoretical machine learning (ensemble bagging, bias-variance tradeoff), data leakage prevention, REST API architectural design, runtime schema validation with Pydantic, and modern reactive frontend development with Angular 21 and RxJS.

8.3 Identified Limitations & Challenges

- **Class Imbalance at Score Tails:** True quality ratings in the UCI dataset are concentrated around 5 and 6 (82.5% of samples). Extreme scores (3 and 8) have scant representation (< 2% combined), restricting tail-end sensitivity.
- **Human Sensory Noise Floor:** Human sensory ratings inherently carry inter-evaluator variance (~0.5 rating points). Consequently, residual errors near 0.42 reflect natural sensory variance rather than model deficiency.
- **Omitted Enological Factors:** Crucial chemical drivers — such as tannin extractability, polyphenols, and grape varietal — are absent from the basic 11-feature benchmark.

8.4 References & Academic Bibliography

Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. Decision Support Systems, 47(4), 547-553.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

Ramirez, S. (2024). FastAPI: High performance, easy to learn, fast to code. Retrieved from https://fastapi.tiangolo.com/

Google. (2024). Angular Documentation — Modern Web Development Framework. Retrieved from https://angular.dev/

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51-56.